Week 9 DevSecOps Evidence

Repository: jakeryderv/devops-week9
Issue: Week 9 - DevSecOps Security Hardening #1
Pull request: Complete Week 9 DevSecOps security hardening #2
Branch: `security/week9-devsecops`
Date: October 30, 2026

The Help Desk application was checked before and after non-root hardening. The original CI and new security workflow both passed on implementation commit `9322bd7` and evidence commit `608a05f`.

Milestone 1 - Baseline application and tests

Milestone 1 - Baseline application and tests

Saved command transcript | 01-baseline-tests.txt

```
Captured UTC: 2026-10-30T15:54:37.975176+00:00
$ .venv/bin/python -m pytest -v

============================= test session starts ==============================
platform linux -- Python 3.13.11, pytest-8.4.2, pluggy-1.7.0 -- /home/jake/school/devops/week9/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/jake/school/devops/week9
collecting ... collected 2 items

tests/test_db.py::test_create_and_list_request PASSED                    [ 50%]
tests/test_db.py::test_update_status PASSED                              [100%]

============================== 2 passed in 0.30s ===============================

Exit code: 0
```

Both baseline database tests passed before Dockerfile hardening, and the Compose stack ran the web, PostgreSQL, and Adminer services with a healthy database.

Tests | Compose services | Application check

The starter used the old PostgreSQL volume path with `postgres:18`; the mount was corrected to `/var/lib/postgresql` before baseline startup. The application image still ran as root during this baseline.

Milestone 2 - .env protected from Git

Milestone 2 - .env protected from Git

Saved command transcript | 02-env-protection.txt

```
Captured UTC: 2026-10-30T15:53:51.949802+00:00
$ sh -c 'git check-ignore .env; git ls-files .env; git status --short --branch'

.env
## security/week9-devsecops
 M compose.yaml
?? docs/evidence/

Exit code: 0
```

Git ignores `.env`, and the tracked-file check returns no `.env` entry.

Command output

The local file contains a generated development password; `.env.example` retains its placeholder, and `.dockerignore` also excludes `.env`.

Milestone 3 - Dependabot configured

devops-week9 / .github / **dependabot.yml**

jakeryderv Configure Dependabot and fix PostgreSQL 18 volume mount — 2e49252 · 25 minutes ago — History

Code | Blame — 11 lines (10 loc) · 206 Bytes — Raw

```yaml
version: 2
updates:
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "weekly"

  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

The configuration on GitHub schedules weekly updates for pip and GitHub Actions at the repository root.

Configuration on GitHub

Milestone 4 - Baseline Docker Scout CVE scan

Milestone 4 - Baseline Docker Scout CVE scan

Saved command transcript | 04-scout-cves.txt | Summary excerpt; full output linked

```
Captured UTC: 2026-10-30T15:55:16.568328+00:00
$ docker scout cves helpdesk:week9-before

## Overview

                    |                Analyzed Image
--------------------|---------------------------------------------------
 Target             |  helpdesk:week9-before
   digest           |  fee55445a97a
   platform         | linux/amd64
   provenance       | https://github.com/jakeryderv/devops-week9.git
                    |  https://github.com/jakeryderv/devops-week9/blob/2e4925292a6ba544bf4d2eae78c0b557e74c356a
   vulnerabilities  |    5C    42H    23M    28L
   size             | 200 MB
   packages         | 186
                    |
 Base image         |  python:3.13-slim
                    |  aaf08c5490d5


## Packages and Vulnerabilities

   5C    35H    18M     0L  pillow 11.3.0
pkg:pypi/pillow@11.3.0
```

Docker Scout scanned `helpdesk:week9-before` and reported 98 vulnerabilities: 5 critical, 42 high, 23 medium, and 28 low.

Complete CVE output | Baseline build | Baseline UID 0

Scout indexed 186 packages and identified 19 vulnerable packages. Pillow 11.3.0 accounted for all five critical findings and 35 high findings; these are scanner findings, not proof that the Help Desk exposes an exploitable path for every CVE.

Milestone 5 - Docker Scout recommendations / SBOM evidence

Milestone 5 - Docker Scout recommendations / SBOM evidence

Saved command transcript | 05-scout-recommendations.txt | Summary excerpt; full output linked

```
Captured UTC: 2026-10-30T15:56:14.889270+00:00
$ docker scout recommendations helpdesk:week9-before
 Target   |  helpdesk:week9-before
   digest |  fee55445a97a

## Recommended fixes

  Base image is  python:3.13-slim

 Name            |  3.13-slim
 Digest          |  sha256:aaf08c5490d52cd92bc27142a5215b1e47355a5eaf9adfad2a0fbc022d0b5490
 Vulnerabilities |    0C     7H     3M    27L
 Pushed          | 2 days ago
 Size            | 43 MB
 Packages        | 142
 Runtime         | 3.13.18


Refresh base image
  Rebuild the image using a newer base image version. Updating this may result in breaking changes.

  ✓ This image version is up to date.
```

Scout identified `python:3.13-slim` as the base image and reported that its current image version was up to date despite remaining vulnerability findings.

Recommendations | Software inventory

The base image had 0 critical, 7 high, 3 medium, and 27 low findings. Scout listed newer Python slim tags with the same base-image counts and an Alpine option with fewer base-image findings; a base-image change would require compatibility testing. The SBOM inventories package names, versions, and ecosystems so Scout can match them to vulnerability records. Functional tests do not perform that inventory or CVE comparison.

Milestone 6 - Dockerfile hardened to non-root



devops-week9 / Dockerfile

jakeryderv Run application as non-root and enforce security policies ✓ 9322bd7 · 23 minutes ago History

Code Blame 18 lines (11 loc) · 441 Bytes Raw

```
1   FROM python:3.13-slim
2
3   WORKDIR /app
4
5   COPY requirements.txt .
6   RUN pip install --no-cache-dir -r requirements.txt
7
8   COPY app ./app
9
10  EXPOSE 8501
11
12  # A writable working directory also preserves the local SQLite fallback.
13  RUN useradd --uid 10001 --user-group --create-home --shell /usr/sbin/nologin appuser \
14      && chown appuser:appuser /app
15
16  USER appuser
17
18  CMD ["streamlit", "run", "app/main.py", "--server.address=0.0.0.0", "--server.port=8501"]
```

The Dockerfile creates `appuser` with UID 10001 and switches to it with `USER appuser` before starting Streamlit on port 8501.

Dockerfile on GitHub

The user's home directory is created, application files remain readable, and `/app` is writable to preserve the SQLite fallback.

Milestone 7 - Non-root container verification

Milestone 7 - Non-root container verification

Saved command transcript | 07-non-root-uid.txt

```
Captured UTC: 2026-10-30T15:56:12.943835+00:00
$ docker run --rm --entrypoint id helpdesk:week9-after -u

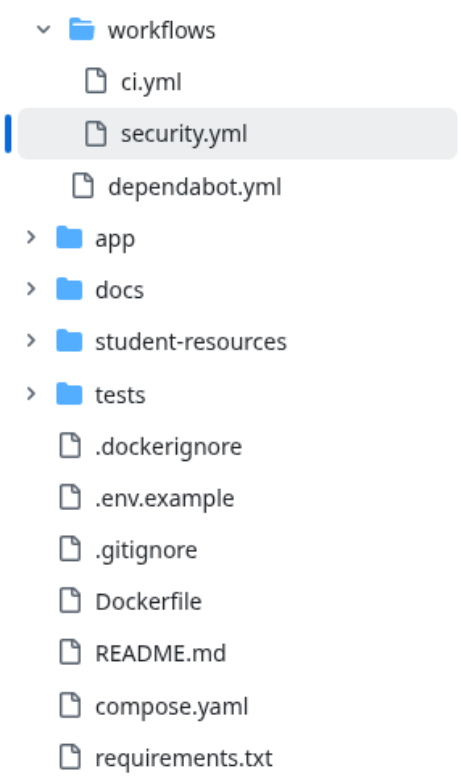
10001

Exit code: 0
```

Running `id -u` in `helpdesk:week9-after` returned 10001, confirming that the hardened image does not run as root.

Image UID | Running service identity | PostgreSQL application check | SQLite fallback | Final tests

Milestone 8 - Security workflow added




```yaml
    branches: [ main ]

permissions:
  contents: read

jobs:
  security-policy:
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository
        uses: actions/checkout@v6

      - name: Reject tracked .env
        run: |
          if git ls-files --error-unmatch .env >/dev/null 2>&1; then
            echo "::error::.env must not be tracked by Git"
            exit 1
          fi
          echo ".env is not tracked by Git"

      - name: Build application image
        run: docker build -t helpdesk:security .

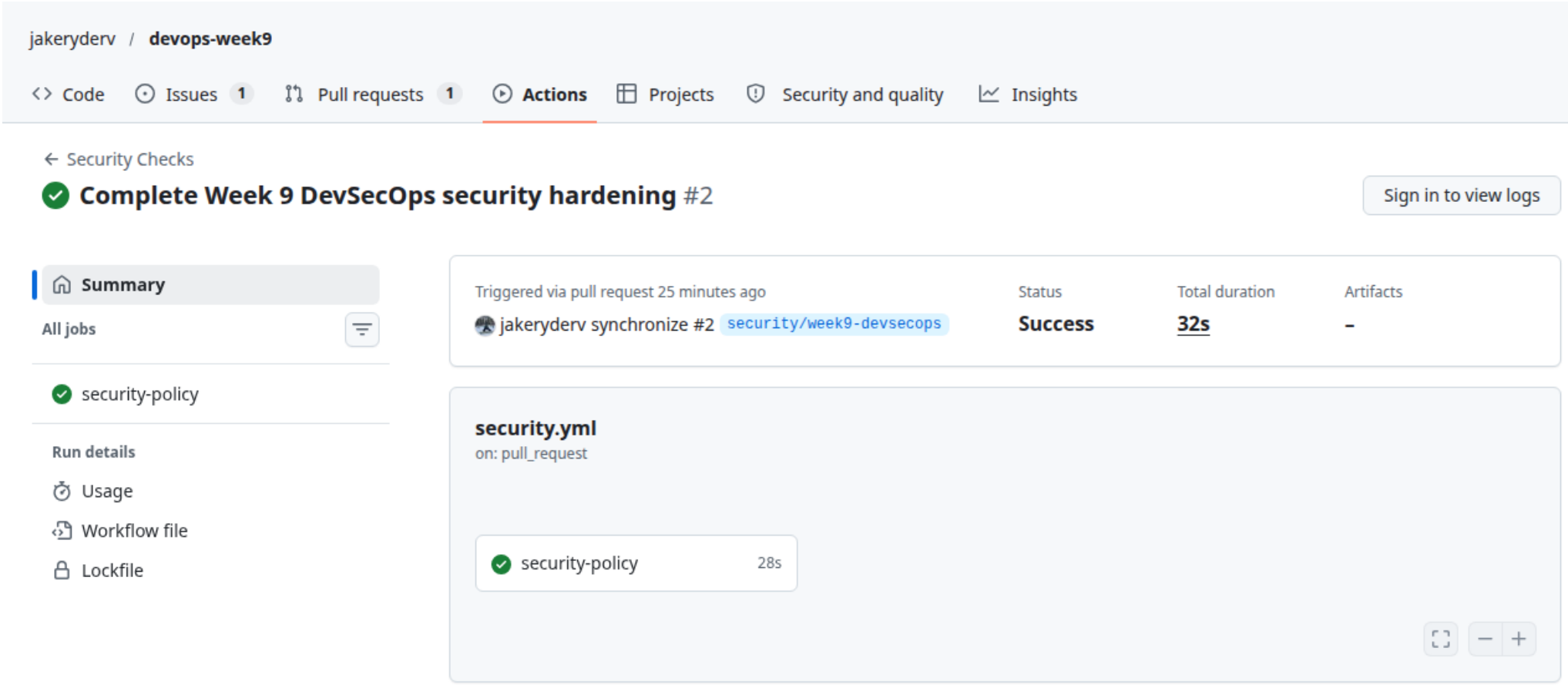
      - name: Verify non-root runtime user
        run: |
          USER_ID=$(docker run --rm --entrypoint id helpdesk:security -u)
          echo "Container UID: $USER_ID"
          test "$USER_ID" -ne 0
```

The GitHub workflow rejects a tracked `.env`, builds the image, and fails when its runtime UID is 0 on pushes and pull requests to `main`.

Workflow on GitHub | Positive and negative policy checks

The exact shell checks accepted the clean repository and UID 10001 image, and rejected a dummy tracked `.env` in a temporary repository and the UID 0 baseline image.

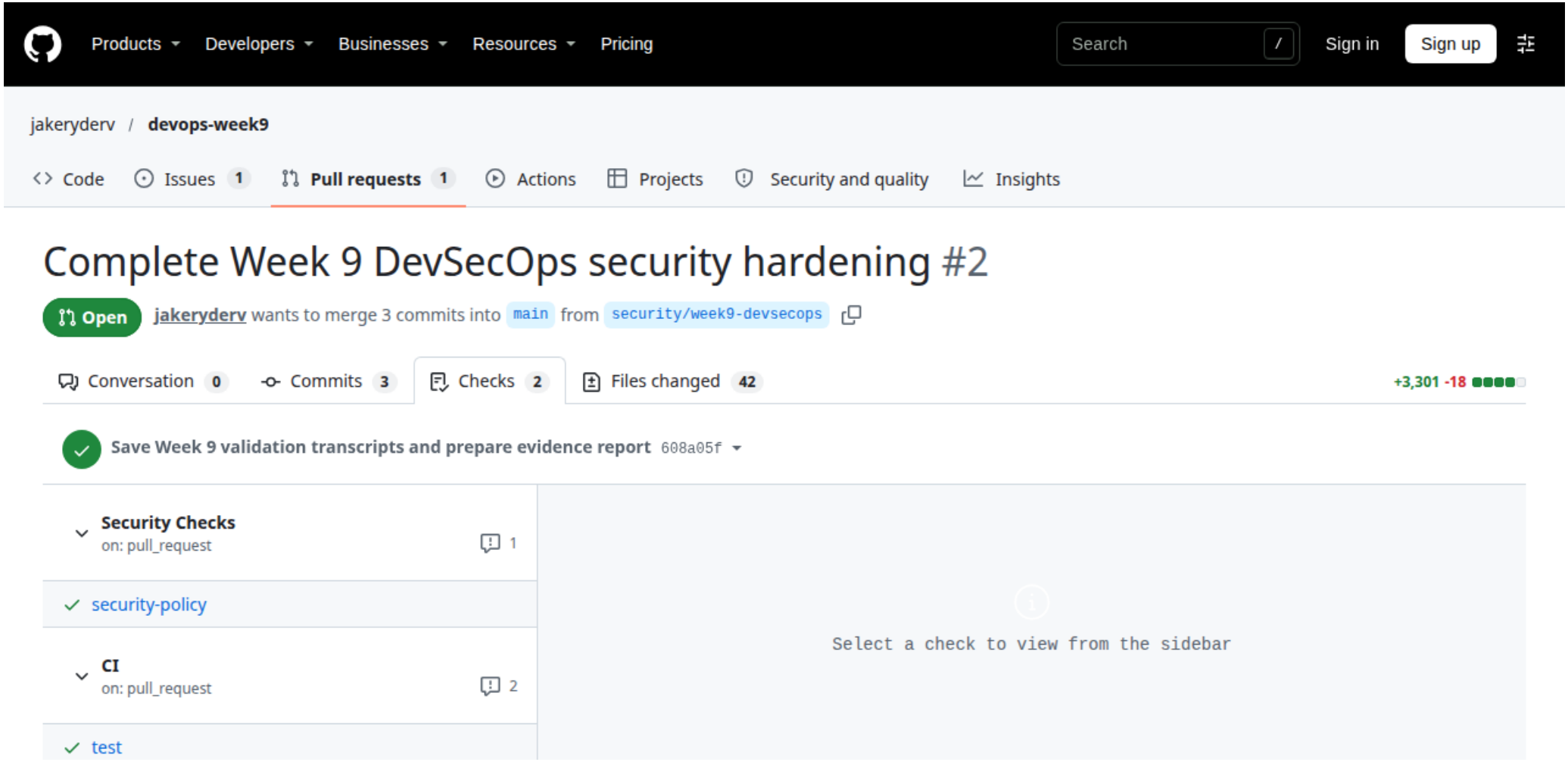
Milestone 9 - GitHub Actions security checks passing



GitHub Actions shows the security-policy job completed successfully for the security pull request.

Security Checks run | CI run | PR check results | Security job log

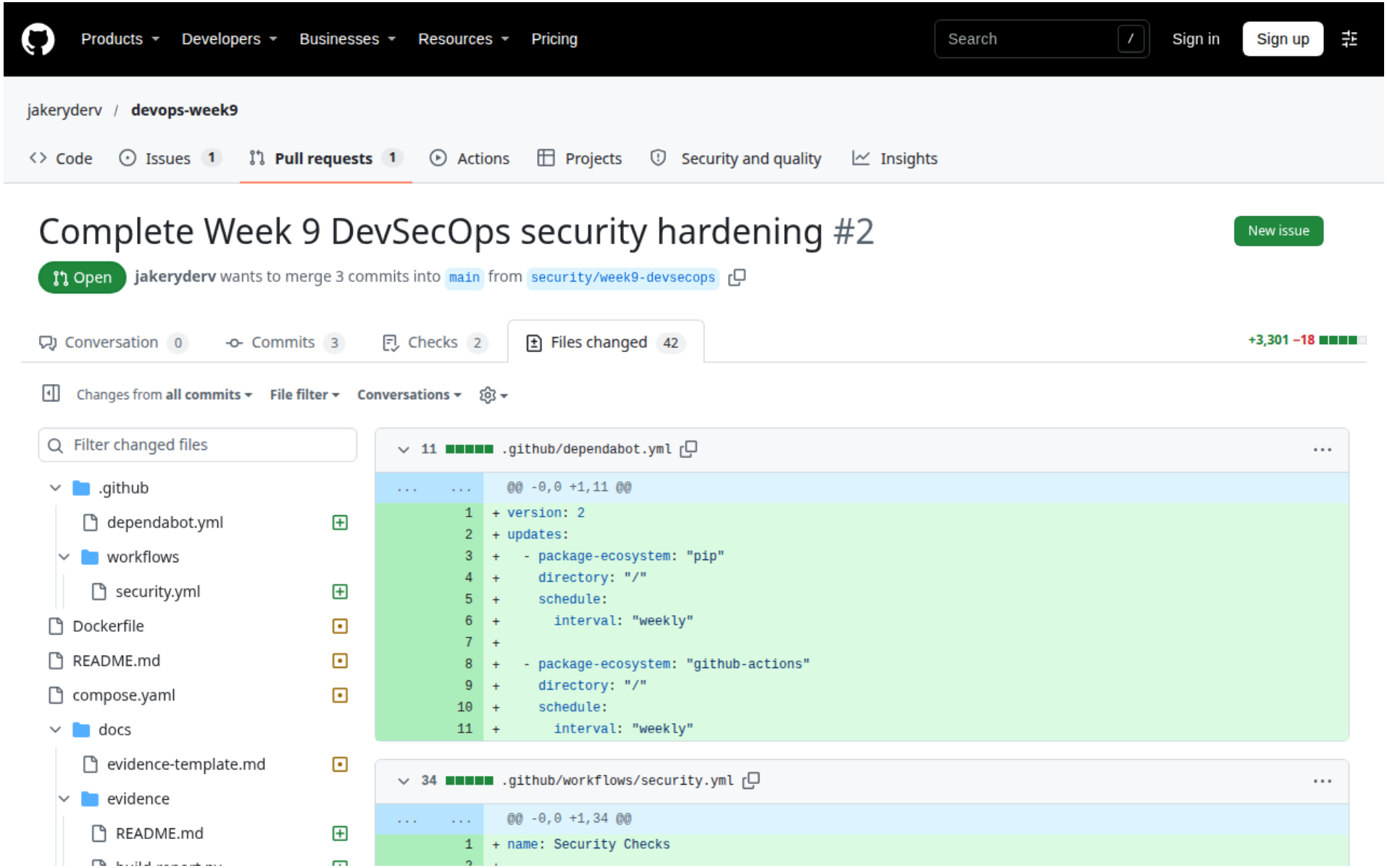
Milestone 10 - Final pull request and reflection



Pull request #2 contains the Week 9 security changes and successful test and security checks.

Pull request

Milestone 10 - Files changed



The pull request includes Dependabot, the security workflow, Dockerfile hardening, the PostgreSQL volume correction, and the supporting evidence.

Milestone 10 - Reflection

What security problem does running a container as non-root reduce?

Running as non-root limits the permissions available if the application is compromised. An attacker using the application process cannot automatically modify root-owned files or perform operations that require root inside the container. It reduces the potential damage but does not remove dependency vulnerabilities or guarantee that a container cannot be escaped.

What did Docker Scout identify that normal functional tests would not identify?

Docker Scout matched installed package versions to known vulnerability records and reported 98 findings in the baseline image, including five critical findings in Pillow. The database tests passed because creating, listing, and updating requests still worked. Those tests do not check the image's operating-system packages or dependency versions against CVE data.

Why is .env.example appropriate for Git while .env usually is not?

`.env.example` documents the variables needed to configure the application using safe placeholder values. The local `.env` contains the actual development password, so committing it would expose that value in the repository and its history. Keeping the example tracked and the real file ignored makes setup repeatable without sharing the secret.

How does the security workflow turn a security expectation into an enforceable pipeline policy?

The workflow turns the expectations into commands that return a failure status when `.env` is tracked or the image runs as UID 0. GitHub Actions runs these checks for pull requests and pushes to `main`, so violations produce a failed pipeline check rather than relying only on manual review. The checks were also tested with deliberately failing inputs to verify that they reject both problems.